

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Catherine Patricia Sindunata - 5024241040

2026

1 Pendahuluan

1.1 Latar Belakang

Mikrokontroler adalah komponen inti dari hampir semua perangkat elektronik modern, mulai dari peralatan rumah tangga, perangkat wearable, sistem otomasi industri, hingga perangkat *Internet of Things* (IoT). Kemampuannya membaca masukan dari lingkungan, mengolahnya, lalu mengendalikan keluaran membuat mikrokontroler menjadi jembatan antara dunia fisik dan dunia digital. Karena itu, mahasiswa perlu menguasai dasar pemrograman mikrokontroler sebelum mempelajari sistem tertanam (*embedded system*) yang lebih kompleks.

Di industri, pengembang jarang bekerja hanya dengan satu jenis *chip* atau satu *framework*. Dua *platform* yang banyak dipakai saat ini adalah STM32 dari STMicroelectronics, yang berbasis ARM Cortex-M dan populer di aplikasi industri, serta ESP32 dari Espressif, yang memiliki WiFi dan Bluetooth terintegrasi sehingga banyak dipakai untuk produk IoT. Masing-masing dapat diprogram dengan *framework* berbeda, seperti Arduino yang sederhana dan ESP-IDF yang lebih *native* dan berbasis FreeRTOS. Pemahaman tentang perbedaan *board*, cara *flashing*, dan karakteristik *framework* ini penting agar praktikan dapat memilih alat yang tepat sesuai kebutuhan. PlatformIO pada VSCode dipakai sebagai lingkungan pengembangan karena mampu menangani berbagai *board* dan *framework* dalam satu antarmuka.

Selain mengenal perangkatnya, praktikan juga perlu memahami cara mengakses GPIO (*General Purpose Input/Output*) dengan benar, karena di sinilah banyak masalah praktis muncul. Pin *input* yang dibiarkan *floating* akan terbaca acak akibat *noise*, sehingga diperlukan resistor *pull-up* atau *pull-down*. Tombol mekanik juga mengalami *bouncing*, yaitu satu kali tekan dapat terbaca sebagai beberapa kejadian, sehingga perlu ditangani dengan *debouncing*. Di sisi lain, banyak sensor menghasilkan sinyal analog yang tidak cukup dibaca sebagai HIGH atau LOW saja, sehingga diperlukan ADC (*Analog-to-Digital Converter*).

Berdasarkan hal tersebut, praktikum ini dilaksanakan untuk memperkenalkan *board* STM32 Blackpill dan ESP32, *framework* Arduino dan ESP-IDF, serta tiga mode akses GPIO, yaitu digital output, digital *input*, dan analog *input*. Melalui percobaan Blink, pembacaan tombol dengan *pull-up/pull-down* eksternal dan internal, *debouncing*, serta pembacaan LDR melalui ADC, praktikan diharapkan memiliki dasar yang kuat untuk modul-modul berikutnya dan untuk pengembangan sistem tertanam di dunia nyata.

1.2 Dasar Teori

1.2.1 PlatformIO dan VSCode sebagai Lingkungan Pengembangan

PlatformIO adalah ekosistem pengembangan *embedded* yang bersifat *cross-platform* dan mendukung ratusan *board* serta berbagai *framework*, seperti Arduino, STM32Cube/HAL, dan ESP-IDF, dalam satu antarmuka yang seragam. Salah satu keunggulannya adalah kemampuan menjalankan *framework* berbeda pada chip yang sama hanya dengan mengubah konfigurasi pada berkas `platformio.ini`. Sebuah project PlatformIO terdiri atas berkas `platformio.ini` yang berisi konfigurasi board, *framework*, dan protokol *upload* (folder `src/` untuk kode program utama; folder `include/`

untuk berkas header; serta folder `lib/` untuk library tambahan).

1.2.2 STM32 Blackpill dan Framework Arduino

STM32 Blackpill adalah *development board* berbasis mikrokontroler STM32F401CC dengan CPU ARM Cortex-M4 32-bit yang dilengkapi FPU dan bekerja pada clock hingga 84 MHz. Board ini memiliki memori Flash 256 KB dan RAM 64 KB, LED onboard pada pin PC13 yang bersifat aktif LOW, serta tombol user pada pin PA0. Blackpill tidak memiliki *programmer onboard*, sehingga proses *flashing* memerlukan ST-Link eksternal melalui protokol SWD (*Serial Wire Debug*) dengan jalur SWDIO, SWCLK, GND, dan 3.3V.

STM32 dapat diprogram menggunakan *framework* Arduino yang dikenal sebagai STM32duino. *Framework* ini menyediakan fungsi yang familiar seperti `pinMode()`, `digitalWrite()`, dan `delay()`, sehingga pemula dapat memprogram STM32 tanpa harus memahami *register* atau HAL secara detail. Meskipun demikian, STM32duino tetap dibangun di atas HAL (*Hardware Abstraction Layer*), yaitu pustaka resmi STMicroelectronics untuk mengakses *peripheral*. Ciri khas STM32duino adalah penamaan pin yang langsung mengikuti label *port* fisik pada *board*, misalnya `PC13` atau `PA0`.

1.2.3 ESP32 dan Framework ESP-IDF

ESP32 adalah mikrokontroler 32-bit *dual-core* berbasis Xtensa LX6 dengan WiFi dan Bluetooth terintegrasi. *Board* ESP32 DevKit sudah dilengkapi konverter *USB-to-Serial*, sehingga dapat diprogram langsung melalui kabel USB tanpa *programmer* eksternal.

ESP-IDF (*Espressif IoT Development Framework*) adalah *framework* resmi dan *native* dari Espressif yang dibangun di atas sistem operasi waktu nyata FreeRTOS. Jika framework Arduino menyederhanakan program menjadi fungsi `setup()` dan `loop()`, ESP-IDF menggunakan struktur berbasis *task* dan *component* dengan titik masuk berupa fungsi `app_main()`. Penundaan waktu pada ESP-IDF dilakukan dengan `vTaskDelay (pdMS_TO_TICKS(ms))`, yang berbasis *tick* FreeRTOS dan tidak memblokir *task* lain. ESP-IDF memiliki tingkat abstraksi lebih rendah dibanding Arduino, tetapi memberi akses langsung ke fitur RTOS seperti *task, semaphore,* dan *queue*, serta konfigurasi *low-level* WiFi. Karena itu, ESP-IDF banyak digunakan pada pengembangan produk IoT tingkat lanjut.

1.2.4 Resistor Pull-up dan Pull-down

Kondisi *floating* merupakan suatu kondisi ketika sebuah tombol tidak ditekan dan pin GPIO tidak terhubung ke level tegangan apa pun. Pada kondisi ini nilai yang terbaca tidak pasti dan dapat berubah acak antara HIGH dan LOW akibat *noise*. Masalah ini diatasi dengan menarik (*pull*) pin ke level tegangan tertentu menggunakan resistor. Pada konfigurasi *pull-up*, pin GPIO dihubungkan ke VCC (3.3V) melalui resistor, umumnya 10 kΩ, sehingga pin terbaca HIGH saat tombol tidak ditekan dan LOW saat tombol ditekan karena terhubung ke GND. Pada konfigurasi *pull-down*, pin dihubungkan ke GND melalui resistor, sehingga pin terbaca LOW saat tombol tidak ditekan dan HIGH saat tombol ditekan karena terhubung ke VCC.

Resistor *pull-up* dan *pull-down* dapat dipasang secara eksternal sebagai komponen fisik pada rangka-

ian, atau diaktifkan secara internal melalui firmware. Pada ESP32 dengan *framework* Arduino, *pull-up* internal diaktifkan dengan `pinMode(pin, INPUT_PULLUP)` dan *pull-down* internal dengan `pinMode(pin, INPUT_PULLDOWN)`. Tidak semua pin ESP32 memiliki resistor *pull* internal. Pin *input-only* GPIO 34-39 tidak memilikinya, sehingga wajib menggunakan resistor eksternal bila dipakai sebagai input tombol.

1.2.5 Debouncing

Kontak mekanik pada tombol atau saklar tidak langsung tersambung atau terputus dengan bersih. Getaran fisik saat kontak bersentuhan atau terlepas menghasilkan beberapa transisi HIGH-LOW dalam waktu yang sangat singkat, fenomena yang disebut *bouncing*. Tanpa penanganan, satu kali penekanan tombol dapat terbaca sebagai beberapa kejadian. *Debouncing* adalah teknik untuk memastikan hanya satu transisi yang sah yang terdeteksi, yaitu dengan menunggu sinyal stabil selama periode tertentu, umumnya 20-50ms, sebelum perubahan status dianggap valid. Pada implementasi berbasis *software*, periode ini biasanya diukur menggunakan fungsi `millis()`. Nilai periode *debounce* yang terlalu kecil membuat *bouncing* masih lolos, sedangkan nilai yang terlalu besar membuat tombol lambat merespons.

1.2.6 ADC sebagai Mode Akses GPIO Ketiga

GPIO dapat digunakan dalam tiga mode akses utama. Mode digital *output* digunakan untuk mengendalikan perangkat seperti LED, sedangkan mode digital *input* digunakan untuk membaca status yang hanya memiliki dua kondisi, yaitu HIGH atau LOW, seperti pada tombol. Mode ketiga adalah analog *input*, yaitu membaca tegangan yang berubah secara kontinu melalui peripheral ADC (*Analog-to-Digital Converter*).

ADC mengubah tegangan analog pada suatu pin menjadi nilai digital yang dapat diolah oleh program. ESP32 memiliki ADC internal dengan resolusi bawaan 12-bit, sehingga menghasilkan rentang nilai 0-4095 untuk tegangan sekitar 0-3.3V. Pada *framework* Arduino, nilai ADC dibaca dengan fungsi `analogRead(pin)`, dan dapat dikonversi menjadi tegangan dengan persamaan

$$V = \text{nilai ADC} \times \frac{3.3}{4095}$$

Tidak semua pin GPIO mendukung ADC. Pada ESP32, pin yang terhubung ke ADC1 (misalnya GPIO 32-39) lebih disarankan dibanding ADC2, karena ADC2 memiliki keterbatasan saat modul WiFi aktif.

Salah satu sensor sederhana untuk mencoba pembacaan ADC adalah LDR (*Light Dependent Resistor*), yaitu resistor yang nilai hambatannya berubah sesuai intensitas cahaya. LDR dirangkai sebagai pembagi tegangan bersama resistor 10 kΩ, lalu titik tengahnya dihubungkan ke pin ADC. Perubahan intensitas cahaya mengubah hambatan LDR, sehingga tegangan keluaran pembagi tegangan juga berubah dan dapat dibaca sebagai nilai digital oleh ADC.

2 Tugas Pendahuluan

1. Jelaskan perbedaan mendasar antara *framework* Arduino dan ESP-IDF dalam memprogram mikrokontroler, termasuk kelebihan dan kekurangan masing-masing.

Jawab: *Framework* Arduino membungkus akses perangkat keras dalam fungsi sederhana seperti `pinMode()`, `digitalWrite()`, dan `delay()`, dengan struktur program `setup()` dan `loop()`. Sedangkan ESP-IDF adalah *framework* resmi Espressif yang berjalan di atas FreeRTOS, dengan titik masuk `app_main()` dan program disusun dalam bentuk *task*. Kelebihan Arduino adalah mudah dipelajari, kodenya ringkas, memiliki *library* yang banyak, dan kodenya cukup mudah dipindah ke *board* lain. Kekurangannya adalah banyak detail tersembunyi, kontrol atas fitur RTOS terbatas, dan `delay()` memblokir eksekusi. Berbeda dengan ESP-IDF memberi akses langsung ke *task*, *semaphore*, *queue*, konfigurasi WiFi tingkat rendah, dan *peripheral* secara penuh, misalnya `vTaskDelay()` yang tidak memblokir *task* lain, sehingga lebih cocok untuk produk IoT yang serius. Kekurangannya adalah kurva belajarnya lebih curam, kodenya lebih panjang, dan hanya bisa dipakai untuk chip Espressif.

2. Mengapa *board* STM32 Blackpill memerlukan *programmer* eksternal (ST-Link) untuk di-*flash*, sedangkan ESP32 dapat diprogram langsung melalui kabel USB? Kaitkan jawaban Anda dengan keberadaan *chip USB-to-Serial*.
   Jawab: STM32 Blackpill tidak memiliki *chip* konverter *USB-to-Serial*, sehingga tidak dapat langsung dihubungkan ke komputer melalui USB untuk pemrograman. Untuk memprogram STM32, diperlukan *programmer* eksternal seperti ST-Link yang menggunakan protokol SWD (*Serial Wire Debug*) untuk mengakses memori dan *flash* mikrokontroler. Sebaliknya, ESP32 DevKit sudah dilengkapi dengan *chip* konverter *USB-to-Serial*, sehingga dapat diprogram langsung melalui kabel USB tanpa memerlukan *programmer* eksternal. *Chip* ini mengubah sinyal USB dari komputer menjadi sinyal serial yang dapat dimengerti oleh mikrokontroler ESP32, memungkinkan proses *flashing* dan komunikasi.

3. Jelaskan perbedaan kondisi logika (HIGH/LOW) pada konfigurasi *pull-up* dan *pull-down* resistor, baik saat tombol tidak ditekan maupun saat ditekan.
   Jawab: Pada konfigurasi *pull-up*, pin GPIO dihubungkan ke VCC melalui resistor, sehingga saat tombol tidak ditekan, pin terbaca HIGH karena terhubung ke VCC. Saat tombol ditekan, pin terhubung ke GND, sehingga terbaca LOW. Sebaliknya, pada konfigurasi *pull-down*, pin GPIO dihubungkan ke GND melalui resistor. Saat tombol tidak ditekan, pin terbaca LOW karena terhubung ke GND. Saat tombol ditekan, pin terhubung ke VCC, sehingga terbaca HIGH.

4. Jelaskan fenomena *bouncing* pada tombol mekanik — mengapa hal ini bisa terjadi — dan sebutkan minimal dua pendekatan untuk mengatasinya.
   Jawab: *Bouncing* terjadi karena kontak mekanik pada tombol atau saklar tidak langsung tersambung atau terputus dengan bersih. Getaran fisik saat kontak bersentuhan atau terlepas menghasilkan beberapa transisi HIGH-LOW dalam waktu yang sangat singkat, sehingga satu kali penekanan tombol dapat terbaca sebagai beberapa kejadian. Dua pendekatan untuk mengatasi *bouncing* dibagi secara *hardware* dan *software*. Secara *hardware*, dapat dilakukan dengan memasang filter RC (resistor dan kapasitor) untuk meratakan sinyal, sering ditambah *Schmitt trigger* agar transisinya bersih, atau memakai IC *debouncer* khusus. Sedangkan secara *software* dilakukan dengan mengabaikan perubahan sinyal sampai sinyal stabil selama periode tertentu (sekitar 20–50 ms) dengan `millis()`, atau membaca pin berkali-kali dan baru menerima status setelah beberapa sampel berturut-turut menunjukkan nilai yang sama.

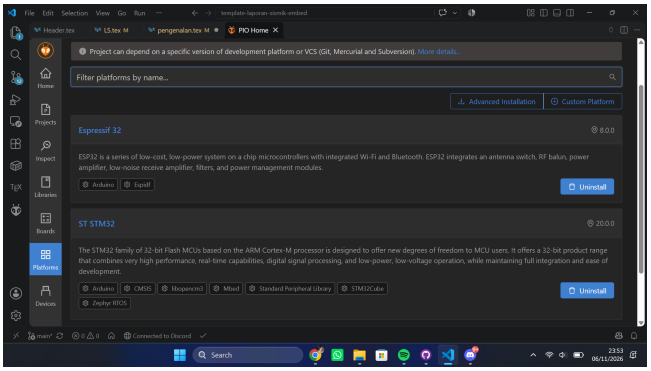
5. Bagaimana ADC (*Analog-to-Digital Converter*) mengubah sinyal analog menjadi nilai digital?

Jelaskan pula pengaruh resolusi bit ADC terhadap presisi hasil pembacaan.

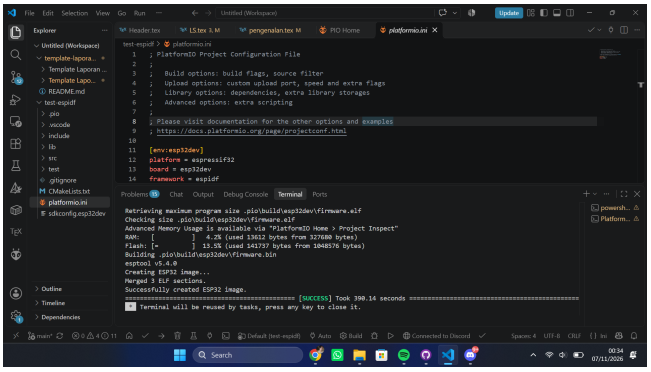
Jawab: ADC mengubah sinyal analog menjadi nilai digital dengan membagi rentang tegangan *input* menjadi sejumlah level diskrit yang sesuai dengan resolusi bit ADC. Misalnya, ADC 12-bit memiliki $2^{12} = 4096$ level, sehingga tegangan *input* dibagi menjadi 4096 langkah. Nilai digital yang dihasilkan berkisar dari 0 hingga 4095, di mana setiap nilai mewakili tegangan tertentu dalam rentang *input*. Semakin tinggi resolusi bit ADC, semakin banyak level diskrit yang tersedia, sehingga presisi pembacaan meningkat. Sebagai contoh, ADC 10-bit hanya memiliki 1024 level, sehingga perubahan tegangan kecil mungkin tidak terdeteksi, sedangkan ADC 12-bit dapat mendeteksi perubahan yang lebih halus dalam sinyal analog.

6. Lakukan *set-up* untuk *tools*: vscode, ekstensi platform-io (dengan *framework* esp-idf dan stm32), dan ekstensi *serial-plotter* pada laptop kalian, berikan *screenshoot* bahwa laptop kalian sudah ter-*set-up* dengan baik.

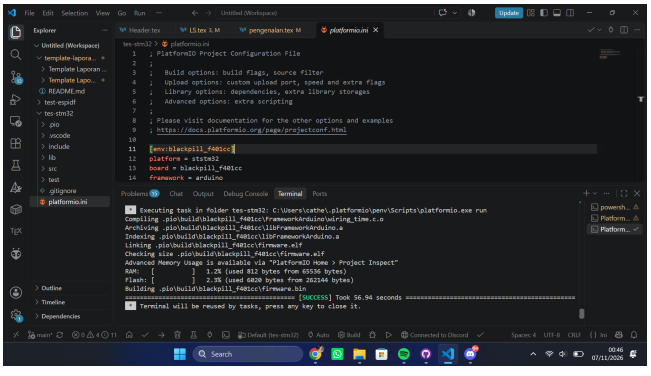
   Jawab:



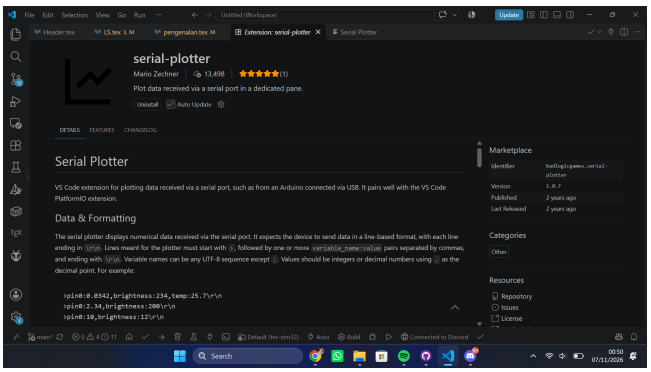
Gambar 1: Platform Espressif 32 dan ST STM32 telah terpasang pada PlatformIO



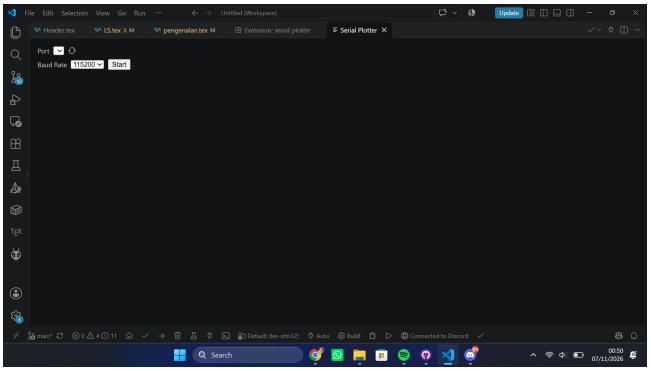
Gambar 2: Hasil build project ESP32 dengan framework ESP-IDF berhasil



Gambar 3: Hasil build project STM32 Blackpill (STM32F401CC) berhasil



Gambar 4: Ekstensi Serial Plotter telah terpasang pada VSCode



Gambar 5: Panel Serial Plotter berhasil dibuka melalui Command Palette